

# Compute-Matched Evaluation of Single- and Multi-Agent LLM Orchestration on Consumer Hardware

Hemadharsana S.<sup>1\*</sup>, Aadithya R.<sup>2</sup> and Midhun S.<sup>2</sup>

<sup>1\*</sup>Department of Computer Science and Engineering (Cyber Security), Saveetha Engineering College, Chennai, 602105, Tamil Nadu, India.

<sup>2</sup>Department of Artificial Intelligence and Machine Learning, Saveetha Engineering College, Chennai, 602105, Tamil Nadu, India.

\*Corresponding author(s). E-mail(s): [hemadharsana@gmail.com](mailto:hemadharsana@gmail.com);  
Contributing authors: [aadithya.ramku@gmail.com](mailto:aadithya.ramku@gmail.com); [midhunsomu39@gmail.com](mailto:midhunsomu39@gmail.com);

## Abstract

Role-based multi-agent large language model (LLM) systems are widely promoted as improving task quality through specialisation, yet two confounds recur in the supporting evidence: multi-agent systems consume more inference compute than their single-agent baselines, and they are often implemented in a different orchestration framework. We report a controlled study addressing both confounds for a fixed small open model on consumer hardware. Holding constant the backbone (Llama 3.1 8B, 4-bit quantised, served locally), the tools, the tasks and the decoding settings, we evaluate four conditions on 132 complexity-stratified tasks (600 execution records): a single-agent ReAct loop, a token-and-call-matched Best-of- $N$  single agent, and a Planner-Researcher-Writer pipeline implemented in both LangGraph and CrewAI. We find an architecture-by-complexity interaction (logistic regression coefficient 1.141,  $p = 0.012$ ): the compute-matched effect size rises from  $-0.164$  at the lowest complexity tier to  $+0.660$  and  $+0.810$  at the two higher tiers. A pre-specified robustness analysis shows that this advantage is driven largely by differential failure rates rather than by answer quality among successful runs, so we report it as a reliability finding. Overhead decomposes into  $+5,359$  tokens ( $+705\%$ ) attributable to architecture and  $+2,748$  tokens ( $+40\%$ ) attributable to framework alone (both  $p < 0.001$ ), and  $78\%$  of attributable pipeline failures arise at inter-agent handoffs. The same model invoked its tools in three conditions and never in the fourth, a property of that framework’s orchestration of a small model. All code, tasks, prompts and raw records are released.

**Keywords:** Multi-agent systems, Large language models, Compute-matched evaluation, Agent orchestration frameworks, Local inference, Edge deployment

## 1 Introduction

Large language model (LLM) agents are increasingly deployed outside cloud environments, on consumer laptops and edge devices where memory capacity, latency and sustained compute are hard constraints rather than billing line items. In these settings the choice between a single-agent execution loop and a role-based multi-agent pipeline is a concrete engineering decision with measurable consequences: a pipeline that adds sequential model invocations to every task multiplies latency and token consumption on hardware with no headroom to absorb either.

The prevailing narrative favours decomposition and specialisation. That narrative derives largely from experiments on large hosted models, where inference cost is elastic and the quality ceiling is high. Two questions are consistently under-controlled in that work, and both matter more, not less, when the backbone model is small and the hardware is fixed.

The first question is whether collaboration is doing the work, or whether additional compute is doing the work. A three-role sequential pipeline issues at least three model invocations per task and re-processes accumulated context at each stage. A single agent given the same total token budget consumes comparable compute through an entirely different mechanism. Unless the baseline is matched on compute, an observed advantage cannot be attributed to collaboration. Chen *et al.* [1] showed that the returns to additional model calls are non-monotonic and difficulty-dependent, which makes the unmatched comparison actively misleading rather than merely imprecise.

The second question is whether measured overhead is architectural or an implementation artefact. Published framework comparisons report multiple-fold differences in token consumption and latency between orchestration frameworks executing identical workflows on identical models [2]. A study that implements its single-agent condition in one framework and its multi-agent condition in another cannot separate the cost of adding agents from the cost of changing frameworks.

We designed a study around both questions and treated a null or negative result as a valid finding throughout. In summary, this research contributes:

- (i) A compute-matched comparison of a role-based pipeline against equivalent single-agent inference on a fixed local 8B-parameter model under a stated memory budget.
- (ii) A same-framework decomposition that separates architecture-attributable from framework-attributable overhead in that regime.
- (iii) A mechanistic account of the observed complexity dependence, grounded in failure-stage annotation and context-growth analysis.
- (iv) Two findings not anticipated by our hypotheses: one pipeline implementation completed every one of 150 executions without failure, and the same model with the same tools invoked them under one framework and never under another.
- (v) A fully released, independently replicable artefact set that requires no cloud compute.

We state plainly that the individual components of this study are each established in prior work. The contribution claimed is their combination under a specific, practically motivated constraint, together with a mechanistic explanation of the observed crossover.

## 2 Related work

### 2.1 Multi-agent failure, compute and role adherence

Multi-agent systems frequently fail to outperform single agents. Cemri *et al.* [3] constructed a taxonomy of multi-agent failure modes (MAST) from a large annotated corpus of execution traces across seven frameworks, identifying fourteen failure modes in three categories. We adopt MAST directly as our failure annotation scheme rather than re-deriving one.

Additional inference calls have non-monotonic returns. Chen *et al.* [1] showed that compound inference systems that aggregate multiple model calls can improve and then degrade as the call count increases, driven by the mixture of easy and hard queries. Li *et al.* [4] showed that simple sampling-and-voting ensembles rival more elaborate collaboration schemes. Together these findings motivate our compute-matched baseline.

Role adherence metrics already exist. Wan *et al.* [5] introduced the Contextualized Role Adherence Score (CRAS), a query-wise metric that decomposes role adherence into four dimensions. We treat CRAS as the reference instrument against which any lightweight role measure must be validated, and we claim no novel metric.

Framework choice imposes substantial overhead. Independent benchmarks that execute identical workflows on identical models report large differences in token consumption and latency attributable to state and context propagation [2]. This is the empirical basis for treating framework as a confound that requires explicit control. Architecture-level comparisons of agent designs were established by BOLAA [6], and role specialisation as a training objective by MALT [7]; our single-agent baseline architecture is ReAct [8].

### 2.2 Differentiation from the closest work

Prajapati and Mohite [9] deployed a five-role multi-agent system on a locally served 7B-class model and compared it against direct prompting and two-call self-refinement on closed-form mathematics and code benchmarks. They concluded that communication format and implementation detail determined outcomes more than architectural complexity. Their baseline is two-call self-refinement rather

than a token-and-call-matched Best-of- $N$ ; they do not control for framework; they use automatically verifiable closed-form benchmarks rather than tool-using, complexity-stratified, open-ended tasks; and they do not measure system-level resource consumption under a stated memory budget. Our results are consistent with their central caution, which we adopt as an explicit hypothesis (H2) rather than an incidental remark.

Orogat *et al.* [2] isolated how framework-level design choices affect performance, robustness, cost and scalability under fixed models, prompts and data. Their benchmark evaluates a heterogeneous framework set on a cloud-served larger model without a memory constraint, does not pair overhead decomposition with a compute-matched quality comparison, and does not perform mechanistic failure-stage analysis. Our second research question is a narrower, hardware-constrained instance of an approach already demonstrated in a different regime, and is positioned as such.

### 2.3 Summary

Table 1 summarises how each body of prior work relates to the present study. Prior work establishes that compute, framework and role adherence each confound multi-agent comparisons, but none controls for compute and framework together on constrained local hardware while also analysing where pipeline failures originate.

**Table 1** Positioning of the present study against prior work

| Prior work                | Contribution                                                                               | Relation to the present study                                                                        |
|---------------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Yao <i>et al.</i> [8]     | Single-agent loop that interleaves reasoning and tool actions (ReAct)                      | Basis of the single-agent baselines (conditions A and B)                                             |
| Chen <i>et al.</i> [1]    | Returns to additional model calls are non-monotonic and difficulty-dependent               | Motivates compute-matched comparison                                                                 |
| Li <i>et al.</i> [4]      | Sampling-and-voting ensembles rival elaborate collaboration schemes                        | Motivates the Best-of- $N$ baseline (condition B)                                                    |
| Cemri <i>et al.</i> [3]   | MAST taxonomy: fourteen failure modes in three categories, seven frameworks                | Adopted as the failure annotation scheme                                                             |
| Wan <i>et al.</i> [5]     | CRAS, a query-wise role adherence metric                                                   | Reference instrument for validating our lightweight role-fidelity proxy                              |
| Liu <i>et al.</i> [6]     | Architecture-level comparison of agent designs                                             | Earlier architecture comparison; no compute matching                                                 |
| Motwani <i>et al.</i> [7] | Role specialisation as a training objective                                                | Roles here are prompt-level charters, not trained                                                    |
| Prajapati and Mohite [9]  | Five-role pipeline versus two-call self-refinement on a local 7B model (closed-form tasks) | We add a matched Best-of- $N$ baseline, framework control, tool-using tasks and resource measurement |
| Orogat <i>et al.</i> [2]  | Unified benchmark of framework-level design choices on a cloud-served larger model         | We pair overhead decomposition with a compute-matched quality comparison under a memory budget       |

## 3 Research questions and hypotheses

We ask four research questions.

*RQ1.* Does a Planner–Researcher–Writer pipeline achieve higher task quality than a token-and-call-matched Best-of- $N$  single agent, and does this difference depend on task complexity?

*RQ2.* What proportion of the multi-agent condition’s overhead is attributable to architecture versus framework?

*RQ3.* Which execution stages account for the variation, and does per-agent output quality degrade with cumulative input context length?

*RQ4.* How faithfully does a 4-bit quantised 8B model preserve assigned roles across handoffs, and does role bleed predict failure?

Each hypothesis is stated so that the opposite outcome is reportable. H1: the compute-matched quality difference is not constant across tiers. H2: a non-negligible proportion of overhead is framework-attributable. H3: per-agent output quality declines as cumulative input context increases. H4: role bleed events are positively associated with downstream task failure.

## 4 Experimental design

### 4.1 Conditions and contrasts

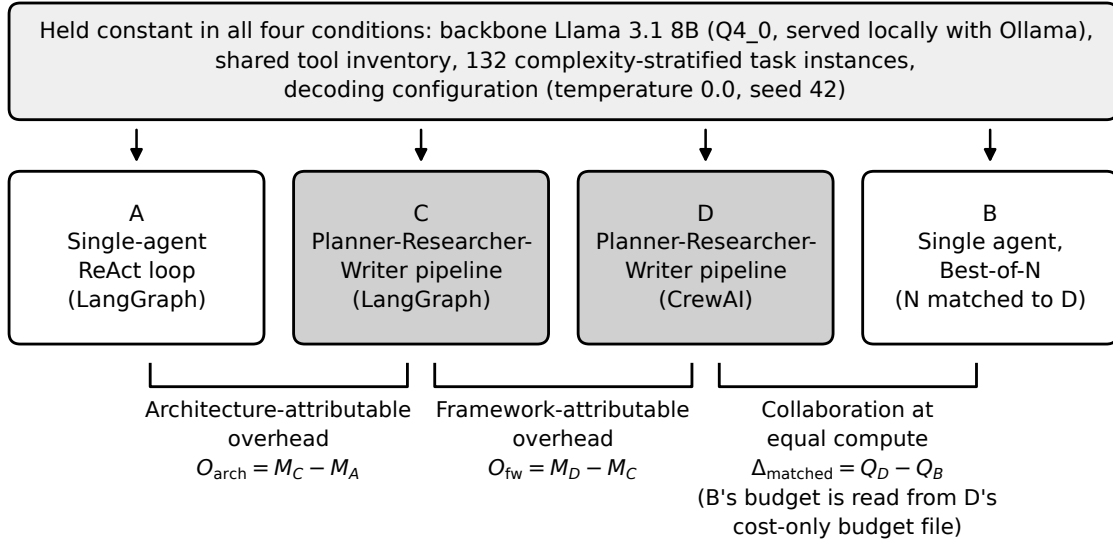
Four conditions share the same backbone model, quantisation, runtime, tool inventory, task instances and decoding configuration; only the execution condition varies (Fig. 1). Condition A is a single-agent ReAct loop [8] in LangGraph. Condition B is the same agent executed  $N$  times with a selection step, where  $N$  is chosen per task so that total tokens and LLM calls fall within a pre-declared tolerance of Condition D’s observed consumption. Condition C is a Planner–Researcher–Writer pipeline in LangGraph. Condition D is the same pipeline in CrewAI. The design yields three contrasts:

$$\Delta_{\text{matched}} = Q_D - Q_B, \quad (1)$$

$$O_{\text{arch}} = M_C - M_A, \quad (2)$$

$$O_{\text{fw}} = M_D - M_C, \quad (3)$$

where  $Q$  denotes task quality and  $M$  a cost measure. The three contrasts measure, respectively, collaboration at equal compute, architecture-attributable overhead and framework-attributable overhead.



Grey boxes: multi-agent pipeline conditions.  $Q$ : task quality.  $M$ : cost measure.

**Fig. 1** Experimental design. All four conditions share the backbone model, tools, tasks and decoding configuration. The three contrasts join neighbouring conditions: architecture-attributable overhead compares A with C, framework-attributable overhead compares C with D, and collaboration at equal compute compares D with B

### 4.2 Prompt parity and compute matching

A single frozen role charter defines each role’s responsibilities and is inserted verbatim into Conditions C and D. The single-agent conditions receive a system prompt that contains the *union* of all three role charters, so that architecture is not confounded with instruction content.

Compute matching is on total tokens and LLM calls within a pre-declared tolerance, with selection cost and retries counted inside the budget. Directionality is enforced in code: Condition D executes first and writes a budget file that contains cost fields only, and a guard rejects any attempt to write a quality-bearing field into it. Condition B therefore cannot observe Condition D’s outcome.

### 4.3 Complexity stratification

Complexity is defined by observable structural factors (Table 2) rather than by author-assigned step counts:

$$C = w_1f_1 + w_2f_2 + w_3f_3 + w_4f_4 + w_5f_5. \quad (4)$$

**Table 2** Complexity scoring and tier definition

| Factor | Operationalisation                  | Weight |
|--------|-------------------------------------|--------|
| $f_1$  | Number of required tool invocations | 1.00   |
| $f_2$  | Number of distinct evidence sources | 1.00   |
| $f_3$  | Dependency depth (chained steps)    | 1.50   |
| $f_4$  | Verification required (0/1)         | 2.00   |
| $f_5$  | Synthesis required (0/1)            | 2.00   |
| Tier 1 | $C \leq 6.25$                       |        |
| Tier 2 | $6.25 < C \leq 13.0$                |        |
| Tier 3 | $C > 13.0$                          |        |

### 4.4 Benchmark

The benchmark comprises 132 tasks, 44 per tier, each carrying a gold fact set and an evaluation rubric. Every task was human-verified for answerability and tier assignment. Tasks draw on a local deterministic document corpus. External web search is deliberately excluded, because it would inject network variance into the latency measurement and make the benchmark irreproducible.

### 4.5 Distributed execution

Execution was distributed across four shards on two physical machine classes. Sharding is by task, not by condition: each shard ran all four conditions on a disjoint task subset, so every paired contrast remains within a single machine and the machine effect cancels inside each difference. Six calibration tasks were executed on every shard to quantify the residual between-machine offset. Configuration hashes were verified identical across all shards before merging, and the merge procedure rejects any task whose conditions span machines.

## 5 Metrics and statistical methodology

Quality is measured by three instruments so that the headline result does not rest on one: deterministic gold-fact coverage (primary), exact match where a task declares one, and an LLM rubric score from a judge of a different model family (secondary). Efficiency, execution, resource and coordination quantities are measured by direct instrumentation; token counts come from the model’s native tokenizer and are never estimated from a proxy. Unmeasurable quantities are recorded as null and never imputed.

The design is paired at the task level. The primary endpoint  $\Delta_{\text{matched}}$  is tested per tier by the Wilcoxon signed-rank test with bootstrap confidence intervals (2000 resamples) and a rank-biserial effect size. H1 is additionally tested with a logistic generalised linear model of task success on condition, tier, their interaction, and machine class as a blocking factor. Three primary hypotheses are Holm-corrected; all other analyses are labelled exploratory with Benjamini–Hochberg false-discovery-rate control.

## 6 Results and discussion

### 6.1 Experimental setup

Six hundred execution records were analysed across 132 tasks and four conditions. Table 3 reports the environment.

### 6.2 Overall comparison

Table 4 shows both pipeline conditions achieving higher mean quality than either single-agent condition. Condition C completed all 150 executions without a single failure, whereas A, B and D

**Table 3** Experimental configuration and environment

| Component          | Specification                         |
|--------------------|---------------------------------------|
| Backbone model     | Llama 3.1 8B                          |
| Quantisation       | Q4_0                                  |
| Runtime            | Ollama (local)                        |
| Temperature / seed | 0.0 / 42                              |
| Context window     | 8192                                  |
| Maximum iterations | 8                                     |
| Judge model        | phi3 (distinct model family)          |
| Machine class 1    | NVIDIA MX550, 2 GB VRAM, 16 GB RAM    |
| Machine class 2    | NVIDIA RTX 3050, 6 GB VRAM, 16 GB RAM |
| Operating system   | Windows 11                            |
| Configuration hash | e4da2c9abab2 (all shards)             |
| Records analysed   | 600                                   |

recorded 43, 50 and 34 failures respectively. Condition D issued no tool calls at all, a framework-level behavioural difference discussed in Sect. 6.9.

**Table 4** Mean values by condition ( $n = 150$  per condition)

| Metric        | A     | B      | C            | D     |
|---------------|-------|--------|--------------|-------|
| Quality       | 0.194 | 0.241  | <b>0.281</b> | 0.280 |
| Success rate  | 0.120 | 0.153  | <b>0.220</b> | 0.167 |
| Total tokens  | 2 691 | 11 849 | 6 897        | 9 437 |
| Input tokens  | 2 296 | 9 826  | 6 214        | 4 542 |
| Output tokens | 396   | 2 023  | 683          | 4 895 |
| LLM calls     | 3.59  | 15.67  | 9.38         | 9.00  |
| Tool calls    | 2.73  | 10.83  | 7.11         | 0.00  |
| Failures      | 43    | 50     | <b>0</b>     | 34    |

### 6.3 Compute-matched comparison (RQ1, H1)

The rank-biserial effect size moves monotonically from  $-0.164$  at Tier 1 to  $+0.660$  at Tier 2 and  $+0.810$  at Tier 3, with Tiers 2 and 3 significant after Holm correction (Table 5, Fig. 2a). Pooled across tiers the effect is not significant ( $p = 0.107$ ), which is expected given that the tier-level effects have opposite signs. The median paired difference is zero at every tier because quality is discrete and most pairs tie; the effect size, not the median, is the informative statistic here.

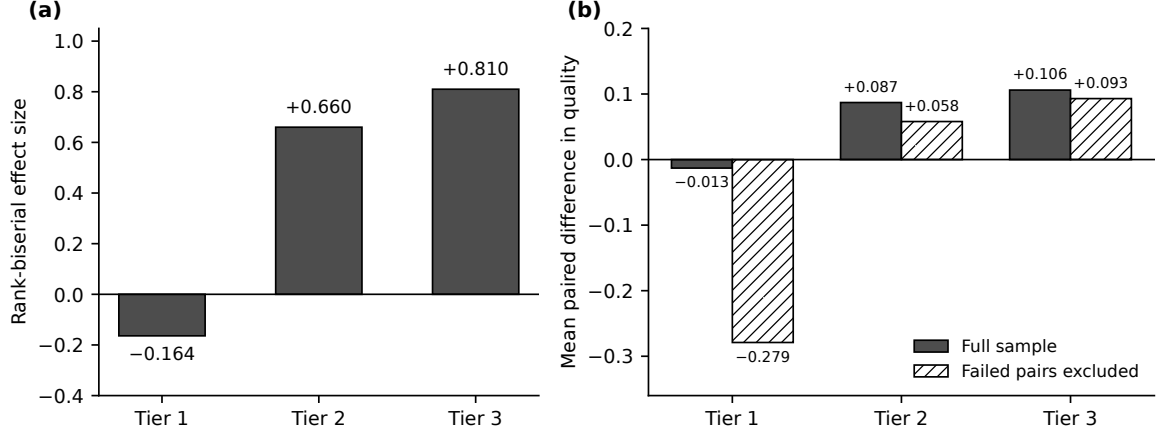
**Table 5** Compute-matched quality difference,  $Q(D) - Q(B)$ 

| Tier   | $n$ | Median | 95% CI         | Effect size | $p$ (Holm)   |
|--------|-----|--------|----------------|-------------|--------------|
| Tier 1 | 44  | 0.000  | [0.000, 0.333] | $-0.164$    | 0.408        |
| Tier 2 | 44  | 0.000  | [0.000, 0.000] | $+0.660$    | <b>0.030</b> |
| Tier 3 | 44  | 0.000  | [0.000, 0.000] | $+0.810$    | <b>0.020</b> |
| All    | 132 | 0.000  | [0.000, 0.000] | $+0.229$    | 0.107        |

The logistic model confirms the interaction directly (Table 6): the condition-by-tier coefficient is 1.141 ( $p = 0.012$ ), so H1 is supported. Machine class is not significant ( $p = 0.746$ ), confirming that task-level sharding removed the hardware effect from the paired contrasts.

**Table 6** Logistic model of task success on condition, tier, their interaction and machine class

| Term                                      | Coefficient   | $p$          |
|-------------------------------------------|---------------|--------------|
| Intercept                                 | +0.555        | 0.361        |
| Machine class (RTX 3050)                  | $-0.105$      | 0.746        |
| Condition                                 | $-1.829$      | 0.025        |
| Tier                                      | $-1.285$      | $< 0.001$    |
| <b>Condition <math>\times</math> Tier</b> | <b>+1.141</b> | <b>0.012</b> |



**Fig. 2** Compute-matched comparison by complexity tier. **a** Rank-biserial effect size for  $Q(D) - Q(B)$ ; positive values favour the CrewAI pipeline. **b** Mean paired difference in quality for the full sample (solid bars) and after excluding pairs in which either condition failed to produce an answer (hatched bars)

## 6.4 Robustness: excluding failed pairs

A failed run scores zero, so a difference in reliability appears as a difference in quality. Table 7 and Fig. 2b repeat the primary analysis using only pairs in which *both* conditions produced an answer.

**Table 7** Full-sample versus failure-excluded  $\Delta_{\text{matched}}$

| Tier   | Full sample |        |       | Failures excluded |        |       |
|--------|-------------|--------|-------|-------------------|--------|-------|
|        | $n$         | Mean   | $p$   | $n$               | Mean   | $p$   |
| Tier 1 | 44          | -0.013 | 0.271 | 14                | -0.279 | 0.047 |
| Tier 2 | 44          | +0.087 | 0.015 | 26                | +0.058 | 0.258 |
| Tier 3 | 44          | +0.106 | 0.007 | 25                | +0.093 | 0.086 |

The direction of the effect survives at every tier, but significance at Tiers 2 and 3 does not. We therefore do not claim that the pipeline produces better answers when both architectures succeed. The defensible claim is that the pipeline is *more reliable* at higher complexity. Conversely, at Tier 1 the compute-matched single agent is significantly better among successful pairs ( $-0.279$ ,  $p = 0.047$ ), which strengthens rather than weakens the crossover interpretation.

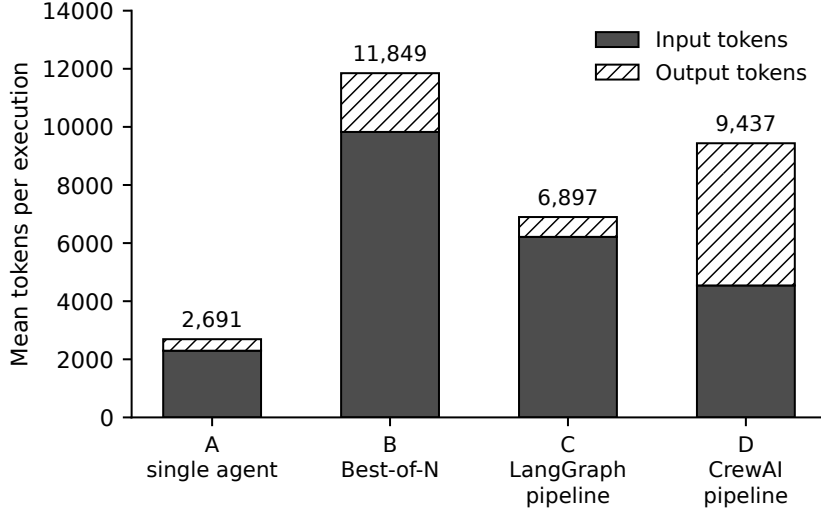
## 6.5 Overhead decomposition (RQ2, H2)

**Table 8** Architecture- and framework-attributable overhead (median paired differences, all  $p < 0.001$ )

| Cost dimension | $O_{\text{arch}}$ (C-A) | %     | $O_{\text{fw}}$ (D-C) | %    |
|----------------|-------------------------|-------|-----------------------|------|
| Latency (s)    | +65.0                   | +88   | +105.6                | +66  |
| Total tokens   | +5 358.5                | +705  | +2 747.5              | +40  |
| Input tokens   | +5 174.5                | +1686 | -1 581.5              | -26  |
| Output tokens  | +287.5                  | +74   | +4 448.5              | +667 |
| LLM calls      | +9.0                    | +900  | -1.0                  | -10  |
| Tool calls     | +7.0                    | —     | -8.0                  | -100 |

Architecture dominates total token cost (+705%), but framework overhead is far from negligible: changing framework at fixed architecture adds 2 748 tokens (+40%) and 105.6 seconds (+66%). H2 is therefore supported (Table 8).

The decomposition is qualitatively different across dimensions (Fig. 3). Adding roles inflates *input* tokens (+1686%) as context is re-processed at each stage, whereas changing framework inflates *output* tokens (+667%) while slightly reducing input tokens. Framework choice therefore does not merely scale cost; it changes its composition. A study that reports only total tokens would miss this entirely.



**Fig. 3** Composition of mean token consumption per execution by condition. Solid segments show input tokens and hatched segments show output tokens; the value above each bar is the mean total

**Table 9** Resource utilisation by machine class. Absolute values must not be pooled across classes

| Class    | Condition | Peak RSS (MB) | VRAM (MB) | Latency (s) | TTFT (s) |
|----------|-----------|---------------|-----------|-------------|----------|
| MX550    | A         | 126.0         | 234       | 121.1       | 8.81     |
|          | B         | 177.6         | 475       | 582.6       | 31.60    |
|          | C         | 151.1         | 214       | 250.2       | 5.69     |
|          | D         | 215.7         | 635       | 573.1       | —        |
| RTX 3050 | A         | 105.5         | 4 110     | 68.8        | 6.33     |
|          | B         | 253.0         | 4 111     | 354.8       | 16.22    |
|          | C         | 125.0         | 4 112     | 121.4       | 4.65     |
|          | D         | 215.7         | 4 146     | 164.8       | —        |

## 6.6 Resource utilisation by machine class

The two machine classes differ qualitatively, not merely in degree (Table 9). On the 2 GB MX550 the quantised model does not fit in video memory (peak VRAM 214–635 MB, that is, context and scratch only), so inference is effectively CPU-bound; on the 6 GB RTX 3050 the model weights are resident (peak VRAM  $\approx$ 4 110 MB) and latency is roughly half. Because sharding is by task, this heterogeneity does not enter the paired contrasts, and machine class is non-significant in the logistic model. Absolute timing figures are nevertheless reported per class and must not be pooled.

## 6.7 Mechanism analysis (RQ3, H3)

**Table 10** Failure stage distribution by MAST category, Tiers 2–3 ( $n = 114$  failed or degraded pipeline executions)

| Originating stage | System design | Misalignment | Verification |
|-------------------|---------------|--------------|--------------|
| Planner           | 0             | 0            | 0            |
| Researcher        | 25            | 0            | 0            |
| Writer            | 0             | 0            | 0            |
| <b>Handoff</b>    | 0             | <b>89</b>    | 0            |

Failures concentrate overwhelmingly at the handoff (Table 10): 89 of 114 attributable failures (78%) are inter-agent misalignment at a handoff boundary, and the remaining 25 are researcher-stage system design failures. No failure originated with the Planner or the Writer. The pipeline’s weakness is therefore not any individual role but the transfer between roles.

For H3, the pooled association between cumulative input context and per-role output quality is strongly negative (Spearman  $\rho = -0.628$ ,  $p < 0.001$ ,  $n = 378$ ). However, this pooled figure must be interpreted with care. Decomposed by role (Table 11), only the Writer shows a within-role decline; Planner and Researcher are flat and non-significant. The Writer also sits at the end of the pipeline with by far the largest accumulated context. The pooled correlation therefore partly reflects



a between-role difference in context volume rather than a uniform within-role degradation. H3 is supported for the Writer stage specifically, not as a general property of every role.

**Table 11** Context growth by role (H3)

| Role          | $n$ | Mean context (characters) | Spearman $\rho$ | $p$               |
|---------------|-----|---------------------------|-----------------|-------------------|
| Planner       | 126 | 130                       | +0.133          | 0.138             |
| Researcher    | 126 | 570                       | +0.105          | 0.244             |
| <b>Writer</b> | 126 | 3 085                     | <b>−0.499</b>   | <b>&lt; 0.001</b> |
| Pooled        | 378 | —                         | −0.628          | < 0.001           |

## 6.8 Role fidelity (RQ4, H4)

**Table 12** Role fidelity score (RFS) by complexity tier (T1–T3) and role bleed rate

| Role       | RFS T1 | RFS T2 | RFS T3 | Bleed (%) | $n$ |
|------------|--------|--------|--------|-----------|-----|
| Planner    | 98.8   | 100.0  | 100.0  | 0.4       | 252 |
| Researcher | 97.6   | 98.2   | 98.8   | 0.4       | 250 |
| Writer     | 87.5   | 79.8   | 84.2   | 0.0       | 252 |

Role adherence is high throughout (Table 12), with the Writer consistently lowest (79.8–87.5) and the Planner near-perfect. Role bleed was observed in only 0.4% of executions. Consequently the H4 test is severely underpowered (Fisher exact test, odds ratio 0.244,  $p = 0.358$ ,  $n = 252$ , with only two bleed-positive executions). H4 is not supported, but the correct reading is inconclusive rather than negative: the phenomenon was too rare in this setting to test. RFS is reported here as an unvalidated lightweight proxy. The CRAS correlation and blind human agreement statistics required to validate it have not been computed, and no metric-level claim is made.

## 6.9 Framework-dependent tool invocation

Condition D issued no tool calls at all (Table 4), while the identical model invoked tools in every other condition: 2.73 calls per task in Condition A, 10.83 in Condition B and 7.11 in Condition C. Because this pattern determines how the  $O_{fw}$  contrast may be interpreted, we investigated it rather than recording it as an unexplained anomaly.

Three checks were performed. First, the tool inventory was confirmed to be correctly bound to the CrewAI agents: three tool wrappers were constructed from the same shared registry used by Conditions A–C, each carrying the identical name and description, and a wrapper executed successfully outside the agent loop with the invocation recorded by the instrumentation. Second, three tool-requiring tasks spanning all three complexity tiers were executed under Condition D; each produced a complete answer and none invoked a tool. Third, because CrewAI dispatches through an intermediate routing layer whose default provider has limited function-calling support, the same task was executed under both the default and the tool-oriented provider configuration. Both produced complete answers and zero tool calls (Table 13).

**Table 13** Tool invocation diagnostic for Condition D on an identical task

| Provider configuration | Tools bound | Tool calls | Tokens | Answered |
|------------------------|-------------|------------|--------|----------|
| Default                | 3           | 0          | 3 891  | Yes      |
| Tool-oriented          | 3           | 0          | 6 852  | Yes      |

We therefore report the absence of tool use in Condition D as a property of the framework’s orchestration of a small model, not as a configuration defect. Under CrewAI’s role and task prompting, the 4-bit quantised 8B model consistently elected to answer from parametric knowledge rather than to invoke available tools, whereas the same model under an explicit ReAct loop (Conditions A

and B) and under the same three-role decomposition in a different framework (Condition C) used those tools readily.

This has a direct consequence for RQ2, stated plainly:  $O_{fw}$  measures the combined effect of framework implementation *and* the tool-use behaviour that framework elicits. These cannot be separated within the present design. We regard the combined quantity as the practically meaningful one for a deployment decision, because a practitioner who adopts CrewAI on this hardware inherits both, but a study that seeks to isolate framework overhead from framework-induced behaviour would require a design in which tool invocation is forced rather than elected.

### 6.10 Failure of repeated sampling at low complexity

Condition B failed on 28 of 50 Tier-1 executions (56%), of which 24 were `no_candidate`: every sampled attempt exhausted the iteration budget without emitting a final answer. On those same 24 tasks, Condition A independently failed 23 times. The tasks are therefore hard for the single-agent architecture as such, and repeated sampling did not rescue any of them.

This is a substantive negative result. It indicates that Best-of- $N$  provides no benefit on tasks whose difficulty lies in *completing* the reasoning loop rather than in the *variance* of its output, which is precisely the regime in which the multi-agent pipeline’s explicit decomposition proved more robust.

## 7 Discussion and deployment guidance

For a practitioner who deploys an agentic system on a 16 GB consumer machine with a small quantised open model, our results support three concrete recommendations.

At low task complexity, do not pay for orchestration. The compute-matched single agent was significantly better among successful Tier-1 pairs, while the three-role pipeline added a median of 5359 tokens per task (Table 8) and returned nothing at this complexity.

At higher complexity, the pipeline’s value is reliability, not eloquence. The pipeline’s advantage at Tiers 2 and 3 came substantially from completing tasks the single agent abandoned, not from producing better answers when both finished. If the deployment requirement is “produce an answer at all”, the pipeline is worth its cost; if it is “produce a better answer”, our evidence does not support paying for it.

Framework choice is a first-class engineering decision. Changing framework at fixed architecture altered token cost by 40%, latency by 66%, and the input/output composition of that cost entirely. Framework selection deserves the same scrutiny that is usually reserved for architecture selection.

The most actionable finding for framework implementers is the failure-stage result (Table 10): 78% of pipeline failures originated at the handoff, not within any role. Effort spent on handoff validation and schema enforcement will yield more than effort spent refining individual role prompts. This is consistent with the inter-agent misalignment category of MAST [3], and our data localise it precisely.

A second unanticipated observation carries similar practical weight. The same model, given the same tools and the same three-role decomposition, used those tools consistently under one framework and not at all under another (Sect. 6.9). Framework choice therefore does not merely alter the cost of an agent system; it can alter whether the agent uses its tools at all. For a practitioner, this is a more consequential difference than any token or latency figure in this paper.

Finally, Condition C’s zero failures across 150 executions, against 34 for the same architecture in a different framework, was not predicted by any of our hypotheses and is, in practical terms, the single largest effect we observed.

## 8 Limitations and threats to validity

**Condition D issued zero tool calls.** As reported in Sect. 6.9, the tool inventory was verified as correctly bound and functional, and the behaviour persisted across two provider configurations. Conditions C and D nevertheless differ in realised tool usage as well as in framework, so  $O_{fw}$  measures framework implementation together with the tool-use behaviour that the framework elicits. We treat the combined quantity as the deployment-relevant one and do not claim to have isolated framework overhead from framework-induced behaviour. A design that forces rather than permits tool invocation would be required to separate them.

**Single backbone model.** Claims are scoped to small local models of this class and are not extended to LLM agents generally.

**Two hardware classes.** The full environment is disclosed; machine class is non-significant in the logistic model, and absolute timing is reported per class.

**Discrete quality scale.** Gold-fact coverage takes few distinct values, producing many tied pairs and zero medians. Effect sizes carry the inferential weight; median differences understate the effect.

**Judge dependence.** The judge is from a different model family than the model under study, and the primary quality measure is deterministic gold-fact coverage rather than a judge score. Judge–human agreement was not validated on a subsample.

**Role metric validity.** RFS is unvalidated, as stated above.

**Failure-driven effects.** Section 6.4 quantifies the extent to which the headline result reflects reliability rather than answer quality; both analyses are reported.

## 9 Conclusion and future work

### 9.1 Conclusion

We asked whether role-based multi-agent collaboration outperforms compute-matched single-agent sampling on a small local model, and how much of its overhead is architectural rather than an artefact of framework implementation. We find a statistically detectable architecture-by-complexity interaction: single-agent sampling is favoured at low complexity and the pipeline at higher complexity. The higher-complexity advantage is driven substantially by differential reliability rather than by answer quality among successful runs, and we report it as such. Overhead decomposes into a large architecture-attributable component and a smaller but substantial framework-attributable component that also changes the composition of cost. Pipeline failures concentrate overwhelmingly at inter-agent handoffs. The numeric crossover location is specific to this model, this quantisation and this hardware class; only its existence and shape should be taken to generalise.

### 9.2 Future work

Future work should resolve the tool-usage asymmetry between framework conditions, validate the role-fidelity proxy against CRAS and human annotation, and extend the capacity axis to test whether the crossover moves with backbone model size.

**Acknowledgements.** The authors thank the Department of Artificial Intelligence and Machine Learning and the Department of Computer Science and Engineering (Cyber Security), Saveetha Engineering College, Chennai, for their support of this work.

## Declarations

**Funding.** [FILL: funding statement is mandatory. If there was no funding, write: “This research received no external funding.” Otherwise give the funder and grant number]

**Competing interests.** The authors declare no competing interests.

**Ethics approval and consent to participate.** Not applicable. The study involved no human participants, animals or personal data.

**Consent for publication.** Not applicable.

**Clinical trial registration.** Not applicable.

**Data availability.** The task set with gold facts, all 600 raw per-task execution records, judging prompts and raw judgements, and all experiment configuration files are available at [https://github.com/Aadithya2222/Research\\_paper](https://github.com/Aadithya2222/Research_paper) and are archived on Mendeley Data at [FILL: Mendeley Data link / DOI].

**Code availability.** Source code for all four conditions, the dependency lockfile, the model identifier, all system prompts and the frozen role charter, and all statistical analysis and plotting scripts are available at [https://github.com/Aadithya2222/Research\\_paper](https://github.com/Aadithya2222/Research_paper) and are archived on Mendeley Data at [FILL: Mendeley Data link / DOI].

**Author contributions.** [FILL: one sentence per author describing their contribution, for example conceptualisation, software, experiments, analysis, writing, supervision. All authors should read and approve the final manuscript]

**Use of generative AI.** Generative AI tools were used to assist with LaTeX formatting and language editing of this manuscript. The authors reviewed and edited all content and take full responsibility for the work.

## References

- [1] Chen L, Davis JQ, Hanin B, Bailis P, Stoica I, Zaharia M, et al. Are more LLM calls all you need? Towards the scaling properties of compound AI systems. Advances in Neural Information Processing Systems (NeurIPS). 2024; <https://doi.org/10.48550/arXiv.2403.02419>.
- [2] Orogat A, Rostam A, Mansour E. Understanding multi-agent LLM frameworks: A unified benchmark and experimental analysis. arXiv preprint. 2026; <https://doi.org/10.48550/arXiv.2602.03128>.
- [3] Cemri M, Pan MZ, Yang S, et al. Why do multi-agent LLM systems fail? arXiv preprint. 2025; <https://doi.org/10.48550/arXiv.2503.13657>.
- [4] Li J, et al. More agents is all you need. arXiv preprint. 2024; <https://doi.org/10.48550/arXiv.2402.05120>.
- [5] Wan G, et al. Diagnose, localize, align: A full-stack framework for reliable LLM multi-agent systems under instruction conflicts. arXiv preprint. 2025; <https://doi.org/10.48550/arXiv.2509.23188>.
- [6] Liu Z, Yao W, Zhang J, et al. BOLAA: Benchmarking and orchestrating LLM-augmented autonomous agents. arXiv preprint. 2023; <https://doi.org/10.48550/arXiv.2308.05960>.
- [7] Motwani SR, Smith C, Das RJ, et al. MALT: Improving reasoning with multi-agent LLM training. Proceedings of the Conference on Language Modeling (COLM). 2025; <https://doi.org/10.48550/arXiv.2412.01928>.
- [8] Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, et al. ReAct: Synergizing reasoning and acting in language models. International Conference on Learning Representations (ICLR). 2023; <https://doi.org/10.48550/arXiv.2210.03629>.
- [9] Prajapati A, Mohite O. Two calls beat five agents: Evaluating multi-agent pipelines against self-refinement for local language models. arXiv preprint. 2026; <https://doi.org/10.48550/arXiv.2607.26922>.